

```
1 // COS30008, Problem Set 4, 2026
2
3 #pragma once
4
5 #include "DoublyLinkedList.h"
6 #include "DoublyLinkedListIterator.h"
7
8 template<typename T>
9 class List
10 {
11 private:
12     using node = typename DoublyLinkedList<T>::node;
13
14     node fHead;
15     node fTail;
16     size_t fSize;
17
18 public:
19
20     using iterator = DoublyLinkedListIterator<T>;
21
22     List() noexcept :
23         fHead(nullptr),
24         fTail(nullptr),
25         fSize(0)
26     {
27
28     }
29
30     ~List()
31     {
32         while (fTail)
33         {
34             fTail->next.reset();
35             fTail = fTail->previous.lock();
36         }
37
38         fHead.reset();
39     }
40
41     List(const List& aOther) :
42         fHead(nullptr),
43         fTail(nullptr),
44         fSize(0)
45     {
46         for (auto iter = aOther.begin(); iter != aOther.end(); ++iter)
47         {
48             push_back(*iter);
49         }
50     }
51 }
```

```
50     }
51
52     List& operator=(const List& aOther)
53     {
54         if (this != &aOther)
55         {
56             List lTemp(aOther);
57             swap(lTemp);
58         }
59
60         return *this;
61     }
62
63     List(List&& aOther) noexcept :
64         fHead(nullptr),
65         fTail(nullptr),
66         fSize(0)
67     {
68         swap(aOther);
69     }
70
71     List& operator=(List&& aOther) noexcept
72     {
73         if (this != &aOther)
74         {
75             swap(aOther);
76         }
77
78         return *this;
79     }
80
81     void swap(List& aOther) noexcept
82     {
83         std::swap(fHead, aOther.fHead);
84         std::swap(fTail, aOther.fTail);
85         std::swap(fSize, aOther.fSize);
86     }
87
88     size_t size() const noexcept
89     {
90         return fSize;
91     }
92
93     template<typename U>
94     void push_front(U&& aData)
95     {
96         node lNew = DoublyLinkedList<T>::makeNode(std::forward<U>(aData));
97         node lNull;
98     }
```

```
99         lNew->link(lNull, fHead);
100
101         if (fHead)
102         {
103             fHead->previous = lNew;
104         }
105         else
106         {
107             fTail = lNew;
108         }
109
110         fHead = lNew;
111         fSize++;
112     }
113
114     template<typename U>
115     void push_back(U&& aData)
116     {
117         node lNew = DoublyLinkedList<T>::makeNode(std::forward<U>(aData));
118         node lNull;
119
120         lNew->link(fTail, lNull);
121
122         if (fTail)
123         {
124             fTail->next = lNew;
125         }
126         else
127         {
128             fHead = lNew;
129         }
130
131         fTail = lNew;
132         fSize++;
133     }
134
135     void remove(const T& aElement) noexcept
136     {
137         node lCurrent = fHead;
138
139         while (lCurrent)
140         {
141             if (lCurrent->data == aElement)
142             {
143                 if (lCurrent == fHead)
144                 {
145                     fHead = lCurrent->next;
146                 }
147             }
148         }
149     }
```

```
148         if (lCurrent == fTail)
149         {
150             fTail = lCurrent->previous.lock();
151         }
152
153         lCurrent->isolate();
154         fSize--;
155
156         return;
157     }
158
159     lCurrent = lCurrent->next;
160 }
161 }
162
163 const T& operator[](size_t aIndex) const
164 {
165     node lCurrent = fHead;
166
167     for (size_t i = 0; i < aIndex; i++)
168     {
169         lCurrent = lCurrent->next;
170     }
171
172     return lCurrent->data;
173 }
174
175 iterator begin() const noexcept
176 {
177     return iterator(fHead, fTail).begin();
178 }
179
180 iterator end() const noexcept
181 {
182     return iterator(fHead, fTail).end();
183 }
184
185 iterator rbegin() const noexcept
186 {
187     return iterator(fHead, fTail).rbegin();
188 }
189
190 iterator rend() const noexcept
191 {
192     return iterator(fHead, fTail).rend();
193 }
194 };
```